



Payatu Case Study

Eliminating Manipulation and Preserving Customer Data in Skill-Based Fantasy Gaming

Project Overview

Skill-based fantasy gaming has shifted from niche to mainstream, handling real money, verified identities, and high-velocity transactions at national scale. Growth has outpaced controls across many stacks, leaving gaps in mobile clients, APIs, and backend services that adversaries exploit for account takeover, payment abuse, and game-integrity manipulation. With a huge user base, the bar is trust, fairness, and compliance in equal measure.

A market-leading operator engaged Payatu to evaluate two flagship fantasy gaming applications. The mandate was clear: identify weaknesses that could expose user data or funds, undermine contest integrity, or threaten platform availability.



The Scope

Payatu defined a focused, risk-aligned scope across client applications and exposed surfaces that directly impact user data, funds, and game integrity.



Mobile application penetration testing

Platforms:

Android

iOS



Web application penetration testing

Process

Mobile PT Process (Android + iOS)



Discovery

The discovery process begins with information gathering, which is one of the most critical stages in a security assessment. This involves identifying the application platform, the frameworks and technologies used, API routes, and third-party integrations within the application.



Understanding the Platform

The [Bandits](#) invest time in understanding the mobile application from an external perspective. This includes studying the company behind the app, its business model, and the stakeholders involved. This context helps frame the security assessment in terms of real-world business impact.



Assessment

Mobile application testing requires a distinct approach, evaluating the app both pre and post-installation to uncover hidden security flaws.



Static Analysis - Android

Static analysis examines the application without executing it. APKs are decompiled using tools such as JADX and APKTool; the resulting code

and AndroidManifest are reviewed for hardcoded secrets, tokens, insecure configuration, exposed components, and third-party libraries to map the app's attack surface and find potential vulnerabilities.



Static Analysis – iOS

Static analysis inspects the .ipa without running it. The .ipa is pulled (e.g., via Frida-ios-dump) and unpacked to extract the app bundle and Mach-O binaries. Tools such as class-dump/class-dump-swift, Hopper, Ghidra, and MobSF are used to recover class/method info and review Info.plist, entitlements, URL schemes, and linked SDKs for sensitive permissions, hardcoded secrets, and risky configurations.



Reverse Engineering – Android

In this phase, the Bandits aim to understand internal code logic and structure without executing it. The APK files are extracted from the device and decompiled using tools such as APKTool, JadX. The resulting code is analyzed for hardcoded secrets, API keys, and logical flows. If Runtime Application Self-Protection (RASP) mechanisms are implemented, their configuration and robustness are examined to identify potential bypass opportunities.



Reverse Engineering – iOS

Testers unpack the .ipa, inspect Mach-O binaries with class-dump/class-dump-swift, Hopper, Ghidra, IDA, otool and nm (with Swift demangling), and review Info.plist, entitlements, URL schemes, , keychain groups, and linked frameworks/SDKs. Analysis checks for hardcoded secrets, ATS misconfigurations, insecure local storage, symbol stripping/debug symbols, , anti-tamper/hardening, and evaluates RASP, jailbreak detection, runtime hooks (method swizzling), dynamic frameworks, and embedded dylibs.



Local File Analysis

Once the application is installed on the device, the Bandits log in using provided credentials and interact with the app as a typical user would. During this interaction, the app writes data to its sandbox environment. The contents of this sandbox such as database files, shared preferences or plist files, cached data, logs, and temporary files are analyzed for insecure data storage practices or exposure of sensitive information like tokens, credentials, or personally identifiable information (PII).



Dynamic Analysis

Dynamic analysis is conducted while the application is running. This includes monitoring local I/O operations during runtime and analyzing network traffic between the app and its backend services. This phase helps uncover runtime vulnerabilities that may not be visible through static inspection alone.



Exploitation

Building on the insights gathered in earlier stages, the exploitation phase involves safely leveraging identified vulnerabilities. The Bandits attempt to gain unauthorized access or bypass security controls such as root/jailbreak detection or certificate pinning to perform restricted actions via the app's APIs. The objective is not just to confirm the existence of a vulnerability but to validate its real-world impact, while staying within the boundaries of the agreed testing scope.



Reporting

This is the final phase, where all findings are consolidated into a clear and structured document for stakeholders. The report includes an executive summary that highlights the overall security posture, key risks, and business impact for non-technical audiences, followed by a detailed technical section for developers and security teams. Each finding is documented with supporting evidence, and all exploitable vulnerabilities in the target system are recorded with associated CVSS v3.1 scores and reported to the client.

Challenges

In-app defenses

The applications implemented shielding controls, and the web surface was protected by a WAF. These defenses constrained test execution and required calibrated request patterns to avoid blocking.



Production build in a controlled environment

Only the production version was shared for testing. Assessment activities were limited to a controlled setup with predefined access.



Geofencing

Access was restricted by geography. Test planning accounted for location controls to ensure legitimate traffic reached the targets within the permitted regions.



Findings

CRITICAL

- Rewards and Currency Manipulation
- Tampering Leaderboard and Season Ranking



HIGH

- Bypass Trial Limit to Watch any Live Content for Free
- Business Logic Flaw - Bypass Free Trial Duration
- Limit of Streaming Session Bypass
- Account Ban Bypass via Response Manipulation
- IDOR and Score Manipulation Possible to Win Client-Side Games



MEDIUM

- Improper Session Handling
- Business Logic Bypass
- GraphQL Introspection Query Enabled
- Session Not Invalidated After Logout
- Bypass COD Unavailable for Orders
- Inadequate Session Management
- Equip Unowned Kits
- Business Logic Flaw - Player Purchase
- Insecure Direct Object References (IDOR) - Room Deletion



MEDIUM

- Improper Assets Management – Access to Deprecated Game Asset Files via API
- User Can Access Internal Games
- Block Puzzle Game Manipulation via Frida Hooking
- Multiple API Versions Leak Sensitive Data



LOW

- No Rate Limit Enforced on Generate OTP Operation
- No Rate Limit on Contact Us Form
- Application Vulnerable to Runtime Manipulation
- Improper Session Management
- Improper Error Handling
- Bypassing Stock Holding Duration
- Trading Cool Down Bypass
- Minimum Amount Withdraw Bypass
- Hardcoded Staging URL in Production APK
- Weak SSL/TLS Cipher in Use
- Verbose Error Messages
- Application Vulnerable to Runtime Manipulation
- OTP Reuse
- Hardcoded Sensitive Information
- Cross Origin Resource Sharing (CORS) Misconfiguration
- Jailbreak Detection Not Implemented
- Application Vulnerable to Runtime Manipulation
- Username Enumeration



LOW

- Blind SSRF in Multiple API Requests and Headers
- Weak Input Validation
- Deposit Limit Bypass
- Exposure of Hardcoded Secrets in Android APK Build
- Unencrypted Hermes Code
- Lack of Encryption on plist File
- Partial Code Obfuscation



Business Impact



Unfair wins and prize theft through rewards, currency, score, and leaderboard manipulation, undermining contest integrity



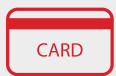
Direct revenue leakage from free trial, streaming limit, and content access bypasses that convert paid experiences into unpaid consumption



Increased fraud velocity due to missing rate limits, weak session controls, OTP reuse, and username enumeration



Reentry of banned or high-risk accounts by bypassing ban logic, driving repeat abuse and higher loss rates



Payment and wallet risk from deposit limit bypass and purchase flow flaws affecting cash flows and reconciliations



Data exposure and IP leakage via IDOR, verbose errors, GraphQL introspection, multiple API versions, hardcoded secrets, and CORS misconfigurations



Service disruption and destructive actions through API abuses like room deletion and query batching, harming availability and gameplay



Elevated cheat development and distribution due to weak client hardening, runtime manipulation, and partial obfuscation



Partner and brand trust erosion as platforms appear insecure or “gameable,” impacting growth, retention, and sponsorships



Compliance and contractual exposure where minors, payments, and personal data are involved, triggering audits and penalties



Higher operating costs from disputes, refunds, customer support escalations, fraud operations, and engineering rework

Before Vs After

BEFORE 	AFTER 
Score, currency, and leaderboard manipulation enabled unfair wins and fraudulent payouts.	Contest logic and access controls are hardened. Illicit score and reward changes are blocked, preserving fair play and protecting payouts. The App was resilient against ranking tampering.
Trial, streaming limit, and live content bypasses turned paid experiences into unpaid consumption.	Trial, session, and content gates are enforced correctly. Paid content cannot be accessed for free, protecting recurring revenue and improving conversion from trial to paid.
Weak sessions, OTP issues, missing rate limits, and username enumeration amplified botting and takeover attempts.	Session lifecycle and OTP flows are enforced, with throttling guards active. Automated abuse drops, account integrity improves, and customers stay protected against takeover.
Banned users reentered via response manipulation, increasing repeat fraud.	Server-side ban checks are authoritative and tamper-resistant. Bad actors cannot reenter, reducing fraud recurrence and support overhead.

BEFORE 	AFTER 
Deposit and purchase logic gaps risked cash leakage and reconciliation errors.	Transaction validation and business rules are enforced at the API. Wallet integrity is maintained, cash flows reconcile cleanly, and revenue leakage is contained.
IDORs, verbose errors, GraphQL introspection, version sprawl, and hard-coded secrets aided attacker recon and leakage.	Object-level access controls, error hygiene, schema hardening, version governance, and secret management are in place. Customer data is safeguarded, and sensitive metadata no longer leaks.
IDOR on administrative actions, batching quirks, and weak input validation enabled room deletion and misuse.	Robust authorization, strict input validation, and rate controls protect critical endpoints. Service stability improves, and destructive actions are blocked.
Partial obfuscation and runtime manipulation lowered the cost to build and maintain cheats.	Stronger anti-tamper and obfuscation increase attacker effort and reduce cheat persistence. Mobile clients resist runtime manipulation attempts.

BEFORE	AFTER
 Findings not risk-ranked, leading to slow mitigation and recurring incidents.	 A risk-ranked remediation roadmap is implemented with owners and verification. Time to fix improves, and regressions are prevented through controlled releases.
Revenue leakage, disputes, and security complaints eroded users, partners, and regulator confidence.	Measurable reductions in abuse, refunds, and complaints. Customer trust recovers, partner confidence strengthens, and the business scales with a secure, compliant posture.

About Payatu

Payatu is a Research-powered cybersecurity services and training company specialized in IoT, Embedded Web, Mobile, Cloud, & Infrastructure security assessments with a proven track record of securing software, hardware and infrastructure for customers across 20+ countries.



Mobile Security Testing [↗](#)

Detect complex vulnerabilities & security loopholes. Guard your mobile application and user's data against cyberattacks, by having Payatu test the security of your mobile application.



Web Security Testing [↗](#)

Internet attackers are everywhere. Sometimes they are evident. Many times, they are undetectable. Their motive is to attack web applications every day, stealing personal information and user data. With Payatu, you can spot complex vulnerabilities that are easy to miss and guard your website and user's data against cyberattacks.



DevSecOps Consulting

DevSecOps is DevOps done the right way. With security compromises and data breaches happening left, right & center, making security an integral part of the development workflow is more important than ever. With Payatu, you get an insight to security measures that can be taken in integration with the CI/CD pipeline to increase the visibility of security threats.

Product Security



Product Security [↗](#)

Save time while still delivering a secure end-product with Payatu. Make sure that each component maintains a uniform level of security so that all the components “fit” together in your mega-product.



Cloud Security Assessment [↗](#)

As long as cloud servers live on, the need to protect them will not diminish. Both cloud providers and users have a shared. As long as cloud servers live on, the need to protect them will not diminish.

Both cloud providers and users have a shared responsibility to secure the information stored in their cloud Payatu’s expertise in cloud protection helps you with the same. Its layered security review enables you to mitigate this by building scalable and secure applications & identifying potential vulnerabilities in your cloud environment.



Code Review [↗](#)

Payatu’s Secure Code Review includes inspecting, scanning and evaluating source code for defects and weaknesses. It includes the best secure coding practices that apply security consideration and defend the software from attacks.



Red Team Assessment [↗](#)

Red Team Assessment is a goal-directed, multidimensional & malicious threat emulation. Payatu uses offensive tactics, techniques, and procedures to access an organization’s crown jewels and test its readiness to detect and withstand a targeted attack.



IoT Security Testing

IoT product security assessment is a complete security audit of embedded systems, network services, applications and firmware. Payatu uses its expertise in this domain to detect complex vulnerabilities & security loopholes to guard your IoT products against cyberattacks.



Critical Infrastructure Assessment

There are various security threats focusing on Critical Infrastructures like Oil and Gas, Chemical Plants, Pharmaceuticals, Electrical Grids, Manufacturing Plants, Transportation systems etc. and can significantly impact your production operations. With Payatu's OT security expertise you can get a thorough ICS Maturity, Risk and Compliance Assessment done to protect your critical infrastructure.



CTI

The area of expertise in the wide arena of cybersecurity that is focused on collecting and analyzing the existing and potential threats is known as Cyber Threat Intelligence or CTI. Clients can benefit from Payatu's CTI by getting – social media monitoring, repository monitoring, darkweb monitoring, mobile app monitoring, domain monitoring, and document sharing platform monitoring done for their brand.

More Services Offered

- AI/ML Security Audit 
- Trainings 

More Products Offered

- EXPLIoT 
- CloudFuzz 



Payatu Security Consulting Pvt. Ltd.

 www.payatu.com

 info@payatu.io

 +91-20-47248026

 +91-8319812123 (SALES)

